Pestaña 1

UNIVERSIDAD ARGENTINA DE LA EMPRESA

Materia: Programación I (Algoritmos y Estructuras de Datos I)
Cuatrimestre/Curso: 2do cuatrimestre, 2025 – Lunes, virtual
Profesor: Abel Israel Laime Huanca

Grupo 3

- Vicente, Zoe Nerea (LU 1210999)
- Pellegrini Chaloum, Francesco (LU 1214900)
- Ferreyra, Martin Gennaro (LU 1216302)
- Cairo, Mariana (LU 1224902)
- Páez De Miguel, Daniela (LU 1222742)

Parcial Integrador

Nombre del proyecto: ***Check & Play***

Índice

Índice	2
Idea general del proyecto	3
Objetivos	3
Definición de roles del equipo	3
Funcionalidades	4
Primer boceto del menú principal (texto o pseudocódigo)	4
Ejemplo inicial de entrada/salida por consola	4
Explicación inicial del código	6
Conclusiones	9
Bibliografía utilizada	9

Idea general del proyecto

Check & Play es una aplicación de consola que permite al usuario gestionar una lista de tareas diarias. Cada vez que completas una tarea, ganas puntos. Estos puntos pueden canjearse por mini-juegos simples durante los momentos de descanso. A medida que se acumulan puntos, se desbloquean diferentes niveles de dificultad, incentivando la productividad mediante la gamificación.

Objetivos

El proyecto **Check & Play** tiene como propósito implementar una aplicación de consola que combine la **gestión de tareas (To-Do List)** con **elementos lúdicos** (gamificación). El sistema busca motivar al usuario mediante un ciclo de *feedback* positivo: completar tareas para ganar puntos, que posteriormente pueden canjearse por acceso a minijuegos simples en momentos de descanso.
A continuación, se presentan los **objetivos de implementación y desarrollo** clave:

- Aplicar los **conceptos fundamentales** de la programación estructurada, tales como el uso de funciones, estructuras de control, listas, módulos y manejo de errores.
- Diseñar un **sistema modular**, donde cada archivo (módulo) cumpla una función específica, favoreciendo la claridad, reutilización y mantenimiento del código.
- Implementar **validaciones de entrada**, asegurando que los datos ingresados por el usuario sean correctos y evitando errores de ejecución.
- Fomentar el **pensamiento lógico** y la resolución de problemas, integrando las distintas partes del programa en un flujo coherente y funcional.
- Desarrollar **documentación clara y ordenada**, describiendo el propósito y funcionamiento de cada módulo para facilitar la comprensión del proyecto por terceros.

Definición de roles del equipo

Product owner: Páez De Miguel, Daniela.
Tech Lead: Cairo, Mariana.
Desarrollador 1: Pellegrini, Francesco.
Desarrollador 2: Ferreyra, Martin Gennaro.
Documentadora: Vicente, Zoe Nerea.

Funcionalidades

Menú principal

1. **Agregar Tarea**: Añade una nueva tarea con validaciones (sin números, mínimo 4 caracteres, no duplicadas).
2. **Completar Tarea**: Marca una tarea como completada y gana puntos.
3. **Ver Tareas**: Muestra todas las tareas con su estado (pendiente o completada).
4. **Canjear Puntos**: Accede a minijuegos si tenés suficientes puntos.
5. **Ver Puntos**: Consulta tus puntos acumulados.
6. **Salir**: Finaliza la sesión.

Mini-juegos disponibles

Juego	Costo (puntos)	Descripción breve
Adivinanza	5	Adiviná un número entre 1 y 10
Trivia	5	Pregunta de cultura general
Batalla naval	10	Atacá una posición para hundir un barco
Memoria	7	Recordá la posición de un emoji
Piedra, papel o tijera	5	Competí contra la máquina

Primer boceto del menú principal (texto o pseudocódigo)

```
--- Menú Principal ---
1. Agregar tarea
2. Completar tarea
3. Ver tareas
4. Canjear puntos
5. Ver puntos
6. Salir
Elige una opción:
```

Ejemplo inicial de entrada/salida por consola

```
--- Menú Principal ---
1. Agregar tarea
2. Completar tarea
3. Ver tareas
4. Canjear puntos
5. Ver puntos
6. Salir
Elige una opción: 1
```

Ingresa la nueva tarea: estudiar algoritmos
✅ Tarea agregada con éxito.
📌 Proceso de agregar tarea finalizado.

--- Menú Principal ---
1. Agregar tarea
2. Completar tarea
3. Ver tareas
4. Canjear puntos
5. Ver puntos
6. Salir
Elige una opción: 3

📝 --- Mi Lista de Tareas ---
1. estudiar algoritmos - [⏳ Pendiente]

💰 Puntos actuales: 0

```
--- Menú Principal ---
1. Agregar Tarea
2. Completar Tarea
3. Ver Tareas
4. Canjear Puntos
5. Ver Puntos
6. Salir
Elige una opción (1-6): 1
Ingresa la nueva tarea: estudiar algoritmos
✅ Tarea agregada con éxito.
📌 Proceso de agregar tarea finalizado.

--- Menú Principal ---
1. Agregar Tarea
2. Completar Tarea
3. Ver Tareas
4. Canjear Puntos
5. Ver Puntos
6. Salir
Elige una opción (1-6): 3

📝 --- Mi Lista de Tareas ---
1. estudiar algoritmos - [⏳ Pendiente]
---
💰 Puntos actuales: 0

--- Menú Principal ---
1. Agregar Tarea
2. Completar Tarea
3. Ver Tareas
4. Canjear Puntos
5. Ver Puntos
6. Salir
Elige una opción (1-6):
```

Explicación inicial del código

El código se organizó en 4 módulos separados para dividir las diversas funciones y fomentar la modularidad, el mantenimiento y la reutilización del mismo:

- **main.py**: Controlador principal del programa. Contiene el bucle de ejecución, el menú de opciones y la inicialización de variables como la lista de tareas y los puntos del usuario.
- **utils**: Carpeta que contiene y organiza los módulos funcionales del sistema, a saber:
    - agregar_tarea.py
    - ver_tareas.py
    - completar_tarea.py
    - canjear_puntos.py
    - ver_puntos.py
- **__init__.py**: Archivo que expone las funciones de los módulos principales contenidas en el paquete *utils*, facilitando su importación desde main.py.
- **files**: Carpeta que contiene el archivos que son utilizados por el proyecto, actualmente dentro se encuentra el archivo juegos.txt que es el que contiene el nombre de los minijuegos y el costo de cada uno.

En ***main.py*** se desarrolló el flujo principal mediante un bucle que presenta un menú al usuario. Este módulo sirve como el punto inicial del programa, manejando la interacción con el usuario y coordinando las llamadas a las funciones que se han importado. También se inician las variables principales, como la lista de tareas y el contador de puntos, dentro de la función main() para prevenir efectos no deseados.

En ***agregar_tarea.py*** se creó una función que pide al usuario que ingrese datos y añade la tarea a la lista en forma de sublista con dos elementos: la descripción y un valor booleano que indica su estado inicial (pendiente). Además, se incluye un bloque try/except/finally para manejar errores en la entrada y asegurar que el proceso sea sólido. También se agregaron validaciones en el ingreso de la tarea que verifica que la tarea no tenga números, tenga un mínimo de 4 caracteres y no estén duplicadas. Se implementó la validación **isdigit()**.

```
--- Menú Principal ---
1. Agregar Tarea
2. Completar Tarea
3. Ver Tareas
4. Canjear Puntos
5. Ver Puntos
6. Salir
Elige una opción (1-6): 1
Ingresa la nueva tarea: pendiente 1
⚠️La tarea no puede contener números.
📌 Proceso de agregar tarea finalizado.
```

En ***ver_tareas.py*** se diseñó la lógica para mostrar los datos, recorriendo la lista mediante un índice manual para presentar cada tarea junto a su estado actual (pendiente o

completada). También se muestra el total de puntos acumulados. Este módulo valida si la lista está vacía, evitando así errores de acceso.

```
--- Menú Principal ---
1. Agregar Tarea
2. Completar Tarea
3. Ver Tareas
4. Canjear Puntos
5. Ver Puntos
6. Salir
Elige una opción (1-6): 3

📝 --- Mi Lista de Tareas ---
1. estudiar algoritmos - [⏳ Pendiente]
2. limpiar la cocina - [⏳ Pendiente]
----------------------------
💰 Puntos actuales: 0
```

En ***completar_tarea.py*** se agregó la opción de elegir una tarea por su índice, validando que exista antes de cambiar su estado. Si se marca la tarea como completada, se suman puntos al contador del usuario. Se implementan validaciones de tipo (ValueError) y control de flujo para asegurar que las operaciones sean seguras y evitar paradas inesperadas.

```
--- Menú Principal ---
1. Agregar Tarea
2. Completar Tarea
3. Ver Tareas
4. Canjear Puntos
5. Ver Puntos
6. Salir
Elige una opción (1-6): 2

📝 --- Mi Lista de Tareas ---
1. estudiar algoritmos - [✅ Completada]
2. limpiar la cocina - [⏳ Pendiente]
----------------------------
💰 Puntos actuales: 10
Ingresa el número de la tarea que completaste: 2
🎉 ¡Felicitaciones! Completaste la tarea y ganaste 10 puntos.
📌 Proceso de completar tarea finalizado.
```

En ***canjear_puntos.py*** se creó la lógica para el canjeo de puntos, comprobando que la cantidad a canjear no supere el saldo disponible a través de excepciones. En un archivo txt se encuentran los mini-juegos con el costo de cada uno. En esta función leemos el archivo y de acuerdo al puntaje puede seleccionar diferentes juegos, una vez que selecciona el juego a canjear se descuenta el costo de los puntos del usuario.

```
--- Menú Principal ---
1. Agregar Tarea
2. Completar Tarea
3. Ver Tareas
4. Canjear Puntos
5. Ver Puntos
6. Salir
Elige una opción (1-6): 4

Tienes 10 puntos disponibles.
Juegos disponibles para canjear:
1. Adivinanza (5 puntos)
2. Trivia (5 puntos)
3. Batalla Naval (10 puntos)
4. Memoria (7 puntos)
5. Piedra, Papel o Tijera (5 puntos)
0. Volver
Elige una opción: 5
✅ Canje realizado: 'Piedra, Papel o Tijera' por 5 puntos.
Te quedan 5 puntos.
```

En ***ver_puntos.py*** se muestra los puntos acumulados por el usuario luego de que éste completó distintas tareas de su lista de pendientes.

```
--- Menú Principal ---
1. Agregar Tarea
2. Completar Tarea
3. Ver Tareas
4. Canjear Puntos
5. Ver Puntos
6. Salir
Elige una opción (1-6): 5

💎 --- Tus Puntos Acumulados ---
⭐ Actualmente tienes 10 puntos.
------------------------------
```

Por otro lado, el archivo ***__init__.py*** se utiliza para consolidar y exponer las funciones principales de los módulos individuales en el paquete utils. Esto permite que las funciones se importen directamente desde el paquete, mejorando así la organización y la claridad del código. Además, la definición de la lista __all__ establece claramente los elementos

públicos del paquete, regulando su visibilidad y el ámbito de importación. Este archivo, por lo tanto, cumple un papel vital como punto de acceso común a las funciones del sistema.

Conclusiones

El desarrollo de Check Play permitió la fusión efectiva de principios básicos de programación estructurada, haciendo uso de funciones, control de flujo, listas y gestión de errores dentro de una arquitectura modular que es comprensible y fácil de mantener. La organización del código en múltiples módulos separados favoreció la reutilización y ayudó a encontrar problemas en una fase temprana, fomentando así la colaboración entre los miembros del equipo.

La implementación de validaciones exhaustivas y un buen manejo de errores aseguró un sistema que es estable, capaz de gestionar entradas incorrectas y que siempre ofrece comentarios claros al usuario. Esta estrategia mejoró notablemente la confiabilidad del programa y mostró buenas prácticas en el desarrollo.

Desde el punto de vista del diseño, el proyecto logró combinar productividad con elementos de juego, transformando una sencilla lista de tareas en una experiencia atractiva e interactiva. Los minijuegos, el sistema de puntuación y las recompensas crearon un entorno ameno que fomenta la constancia y el compromiso de los usuarios con sus objetivos diarios.

En lo que respecta al aprendizaje, el equipo desarrolló habilidades clave como la colaboración, la planificación y la documentación técnica, reconociendo la importancia de la modularidad, la claridad del código y la comunicación eficaz en un proyecto grupal.

Bibliografía utilizada

- Introducción a la programación con Python 3.
- Exposición de experto: Introducción.
- Exposición de experto: Funciones.
- Exposición de experto: Matrices.
- Exposición de experto: Cadena de caracteres.
- Exposición de experto: Excepciones.
- Exposición de experto: Archivos.